SCORING MODELS FOR LLMS

SCORING MODELS FOR LLMS

Typically (1) build a calibrated scoring layer on top of the model's raw outputs, then (2) link different confidence thresholds to business costs (errors, human review, infra) and compare scenarios.[1234]

1. Designing the confidence scoring model

For document classification, don't trust the model's own "I'm X% sure" text; instead, derive confidence from measurable signals and calibrate it. A practical pattern:[567]

1. Generate base signals per document - Use a classifier head if you have one (e.g., fine-tuned model giving class logits). - Or, for a pure LLM API, use: - Logprobs for the chosen label token(s) when available, - Or auxiliary verification (ask a second model to re-classify, or to judge whether the first answer is correct).[89] - Optionally include embedding-level features: cosine distance to class prototypes, margin between top-2 classes, document length, etc.[10] 2. Train a separate confidence model - Build a labeled evaluation set: each item has (document, model's predicted label, gold label). - Compute input features for the confidence model: - Top class probability (softmax of logits or transformed logprobs). - Probability margin between first and second class. - Embedding-based scores (e.g., XGBoost over last-token embeddings as features).[11] - Train a binary model that predicts "was the prediction correct?" and outputs a probability in [0, 1].[1213] - This probability is your raw confidence score for the prediction. 3. Calibrate the raw confidence scores - Use held-out data to calibrate: you want "80% confidence" to mean "about 8 in 10 such predictions are actually correct." - Apply standard calibration methods (Platt scaling, isotonic regression via `CalibratedClassifierCV` in scikit-learn, temperature scaling, etc.).[14] - Validate with reliability diagrams or expected calibration error to see if the confidence scores align with empirical accuracy.[1516] 4. Define operational thresholds - Choose a confidence threshold *t* above which you auto-accept classifications, and below which you fall back to human review, abstain, or escalate to a stronger model.[171819] - Use ROC or precision-recall curves, plus domain risk, to pick

initial thresholds, then refine in production.[202122] - Typical pattern: high-risk domains (legal, financial) run with $t \approx 0.9$–0.95; general back-office workflows might accept $t \approx 0.75$–0.85.[23]

Very simple example:

- Fine-tuned classifier outputs logits → softmax → class probabilities.
- Train a logistic regression on $[p_*\text{top}, p^*_{\text{top}} - p_*\text{second}]$ to predict "correct or not," then calibrate that model.*
- The final calibrated probability per prediction is your ****confidence score****; everything else (thresholds, routing, SLAs) hangs off that.

2. Framing the cost model

Next, you connect a target confidence (or accuracy) level to costs: label data, compute, human review, and downstream error impact.[242526]

Define:

- $C_*\text{label}$: cost per labeled document (internal analyst time or vendor cost).*
- $N_*\text{train}$: number of labeled docs to reach a given accuracy/confidence.*
- $C_*\text{compute}$: cost of training/fine-tuning and inference (GPU hours or API spend).*
- $C_*\text{human}$: cost per human review (per document).*
- $C_*\text{FP}$, C^*_{FN}: business cost of false positives/negatives (e.g., compliance risk, customer impact).
- $p_*\text{auto}(t)$: fraction of documents auto-processed at threshold t.*
- $\text{Err}(t)$: error rate among auto-processed docs at threshold t.

Then:

1. Model build / improvement investment - Data cost: $C_*\text{data} = N^*_{\text{train}} \times C_*\text{label}$. *- Compute cost: estimate GPU/API cost for experiments + final training + ongoing inference, using vendor or infra pricing.*[27] *- Engineering cost: FTE time to build pipelines, evaluation, monitoring (convert to monetary cost if you want a pure financial view).*[28] *2. Operational cost at a given confidence threshold For a volume V docs per period and threshold t: - Human review volume: $V_*review(t) = V \times (1 - p^*_{auto}(t))$.*[29] *- Human review cost: $C_*review(t) = V^*_{review}(t) \times C_*human$.*[30] - Error count among auto-processed docs: $E(t) = V \times p_*\text{auto}(t) \times \text{Err}(t)$.[31] *- Error cost: $C_*error(t) = E(t) \times (w^*_{FP}C_*FP + w^*_{FN}C_*FN)$ where w^*_{FP}, w_*FN are the proportions of error types.* - Infra/API cost: $C_*\text{infra}(t) = V \times C^*_{\text{inference}}$ (plus any extra calls to verification models for low-confidence cases).[32]

Total periodic cost at threshold t:

$$C_*\text{total}(t) = C^*_{\text{review}}(t) + C_*\text{error}(t) + C^*_{\text{infra}}(t)$$

3. Investment to reach higher confidence levels Higher target confidence (e.g., improving from 92% to 97% at your operational threshold) usually requires: - More and better labeled data (active learning, edge cases, error clusters). - Model changes (stronger base model, better fine-tuning, ensembling, verifier models). - Possibly higher-quality infrastructure (faster GPUs, more context, better retrieval).

You estimate the incremental investment ΔC_*build needed to move from your current operating point t_1 to a better one t_2 (or from model A to model B). Then you compare that to the change in periodic costs: - $\Delta C_*period = C^*_{total}(t_2) - C_*total(t_1)$. - If error costs fall significantly (e.g., fewer mis-classified legal documents) or automation increases (fewer reviews), you can quantify payback time and ROI.[33][34]

A simple illustrative calculation:

- Today: 80% auto, 93% accuracy on auto, 20% human review.
- Target: 90% auto, 97% accuracy on auto, 10% review (after investing in better training + calibration).
- Plug your C_*human and C^*_{FP}/C_*FN into the formulas to see annual savings vs the upfront build investment.*

3. Practical workflow to implement this

You can turn the above into a concrete project plan:

1. Establish baseline - Implement initial classifier + simple confidence score (e.g., top-class probability). - Label a test set, compute confusion matrix across thresholds, estimate p_*auto(t) and Err(t).[35][36] - *Quantify current cost per document using your business costs. 2. Build and calibrate the confidence layer - Train the dedicated confidence model on held-out labeled data; calibrate it and validate calibration.[37][38] - Choose initial operating thresholds based on business risk appetite and cost curves.[39][40] 3. Run cost simulations - For a grid of thresholds (e.g., 0.7–0.99), compute: - Automation rate, error rate, expected human-review volume. - Total cost C_*total(t) and net benefit vs human-only baseline.* - Use this to present management a chart: "Here's the cost and risk profile at each confidence level, and the investment required to move from our current point to each improved point."[41][42] 4. Iterate in production - Monitor drift (data and calibration), retrain and recalibrate periodically.[43][44] - Use error cases and low-confidence flags to drive targeted data labeling; this reduces marginal cost of achieving higher confidence over time.[45][46]

Would you be applying this to a high-risk domain (e.g., legal/compliance) or a lower-risk operational workflow? The right thresholds and cost weights depend heavily on that.

⁂

Cross Domain - Risk Analysis

To make it cross-domain and risk-aware, you treat “risk” as an explicit variable alongside “confidence,” and you let that drive both the scoring design and the cost model.[53545556]

1. Add an explicit risk model

You want the system to estimate not just “how likely is this label correct?” but also “how bad is it if this is wrong, given this document and domain?”. A practical setup:[575859]

- Define a small set of risk levels (e.g., low, medium, high) based on business impact, not model uncertainty.
- For each domain/class, attach a default risk level plus rules (e.g., “anything mentioning sanctions, litigation, or patient data is high-risk even if in a ‘generic’ topic”).
- Train or prompt an auxiliary ****risk classifier**** that takes the document (and optionally the predicted label) and outputs a risk level; you can bootstrap labels from policy rules and then refine with human review.[6061]
- At runtime, every prediction is (label, confidence, risk_level).

Example: a marketing email and a regulatory filing may both be classified with 95% confidence, but the latter gets risk=high while the former is low.

2. Risk-conditioned confidence and thresholds

Across domains, calibration and thresholds should depend on risk level, not just class or dataset. Concretely:[62636465]

- Calibrate confidence per risk stratum: fit separate calibration (Platt, isotonic, temperature scaling) for low/medium/high-risk slices so that “0.9” means ~90% accuracy **within that risk band**.[6667]
- Use risk-conditioned thresholds:
  - Low-risk: accept if confidence $\geq t_*$low (e.g., 0.8).*
  - Medium-risk: accept if $\geq t_*$med (e.g., 0.9) else send to human or a stronger model.*
  - High-risk: accept only at $\geq t_*$high (e.g., 0.97) or even force dual-model agreement or human approval.[6869]*
- For cross-domain generalisation, monitor risk-coverage curves (accuracy vs fraction auto-accepted per risk band) to spot where confidence is misaligned as you enter new domains.[7071]

This gives you a “policy surface”: for each (domain, risk_level, confidence) triple, you define the action (auto, escalate, human, refuse).

3. Cost model with risk baked in

Your cost analysis now needs to weight errors, review, and infra by risk and domain, not just overall averages. For a volume V docs in time period T:[72737475]

- Let V_*d, r be docs in domain d and risk level r.*
- For each (d, r) and threshold t_*d, r, estimate from validation data:*
  - $p_*\text{auto}^{d,r}(t)$: fraction auto-accepted.*
  - $\text{Err}^{d,r}(t)$: error rate among auto-accepted.
- Assign costs:
  - $C_*\text{human}^{d,r}$: cost per human review (e.g., legal vs ops).*
  - $C_*\text{err}^{d,r}$: expected loss per wrong classification at that risk/domain (regulatory fine, rework, reputation, etc.).[7677]*

Then for each (d, r):

- Human review volume: $V_*\text{review}^{d,r}(t) = V*_{d,r}(1 - p_*\text{auto}^{d,r}(t*_{d,r}))$.
- Human review cost: $C_*\text{review}^{d,r}(t) = V*_{\text{review}}{}^{d,r}(t) \times C_*\text{human}^{d,r}$.*
- Errors: $E^{d,r}(t) = V_*d, rp*_{\text{auto}}{}^{d,r}(t_*d, r) \times \text{Err}^{d,r}(t*_{d,r})$.
- Error cost: $C_*\text{error}^{d,r}(t) = E^{d,r}(t) \times C*_{\text{err}}{}^{d,r}$.
- Infra/API cost: $C_*\text{infra}^{d,r}(t)$ from per-call cost, including any "second opinion" calls you do at higher risk.[7879]*

Total cost: $C_*\text{total}(t) = \sum*_{d,r}(C_*\text{review}^{d,r} + C*_{\text{error}}{}^{d,r} + C_*\text{infra}^{d,r})$.*

To estimate ****investment required**** to "reach that confidence level", you:

- Choose target operating points t_*d, r^{target} you want per risk band (e.g., high-risk needs 99% empirical accuracy at the accept threshold).*
- From learning curves or pilot experiments, estimate incremental labeled data, engineering work, and infra needed to achieve those accuracies in each domain/risk slice.[808182]
- Compute $\Delta C_*\text{build}$ (extra data + eng + infra spend) and compare to $\Delta C*_{\text{total}}$ (change in annual $C_*\text{total}$ when you move from current thresholds/accuracy to the target risk-aware ones).*

4. Cross-domain, risk-aware implementation steps

An end-to-end flow that respects both cross-domain and risk:

1. Catalog domains and risk taxonomy - Enumerate the document domains you expect (e.g., contracts, support, marketing, finance, HR) and define risk categories with examples for each.[8384] 2. Build joint label–risk dataset - For each sample, annotate: domain, class label, and risk level (from policies, then refined by SMEs). - Ensure you have enough high-risk examples even if they are rare, possibly through targeted sampling.[8586] 3. Train three components - Classifier (LLM-based) for document label. - Risk classifier for risk level from content + context. - Confidence / calibration layer that is conditioned on risk (and optionally domain).[87888990] 4. Estimate risk-conditioned curves - For each (domain, risk), measure accuracy vs confidence threshold, build risk-coverage curves, and derive per-band automation and error behaviour.[9192] 5. Run scenario-based cost simulations - Plug those curves into the cost formulas to simulate different policies (e.g., "what if we relax high-risk threshold from 0.99 to 0.97?") and compute total annual cost and risk.[93949596] 6. Use results to choose investments - Where high risk + poor calibration shows up (e.g., new domain with low feature congruence), invest in domain-specific data, improved calibration, or even specialized models, and quantify whether that pays back in reduced error cost and increased safe automation.[979899100]

Would you want this risk signal to be mostly rule-based from your existing policies, or are you aiming for a learned "risk score" the model infers directly from text across all domains?

⁂

Risk signal would come from rules.

Use the rules engine to generate a ****risk label/score**** per document, then make your LLM's confidence thresholds conditional on that risk and plug them into a risk-weighted cost model.[107108109]

1. Rule-based risk signal

You can treat risk as a deterministic function of content, metadata, and domain. For each document:[110111]

- Run a rules engine (or DSL) that looks at: keywords/phrases, entities (e.g., personal data, financial terms), source system, jurisdiction, customer type.
- Map rules to a discrete risk level and/or numeric risk score, for example:
  - High: contains PII + legal terms + regulated domain.
  - Medium: internal policy, financial summaries.
  - Low: marketing, generic FAQs.
- Keep this auditable: version your rule sets, log which rule fired, and expose that alongside the model's prediction.[112113114]

Output per item becomes: `(domain, label, confidence, risk_level, fired_rules)`.

2. Confidence and policy by risk

Then you use the **same** calibrated confidence model as before, but apply different policies per risk level.[115][116][117]

- Calibrate the LLM's confidence on held-out data once (or per domain), so its probability estimates are well aligned with actual accuracy.[118][119]
- Define risk-dependent thresholds and flows, for example:
  - Low risk: auto-accept if confidence ≥ 0.8; else human if < 0.5; retry/stronger model in between.
  - Medium risk: auto-accept ≥ 0.9; human below.
  - High risk: auto-accept ≥ 0.97 **and** no "critical" rules fired; otherwise human or dual-review.
- Implement this as a simple routing table keyed by `(risk_level, confidence_range)` so behaviour is transparent and explainable to risk/compliance teams.[120][121]

Essentially, rules decide **how bad a mistake would be**; the LLM's confidence decides **how likely a mistake is**; your routing policy combines both.

3. Cost model with rule-based risk

For cost analysis, risk levels from rules determine the cost weights you use for errors and human review.[122][123][124]

For each risk level $r \in \{\text{low, med, high}\}$:

- Estimate, from evaluation data, at your chosen threshold t_r:
  - $p_*\text{auto}^r(t_r)$: fraction auto-accepted.*
  - $\text{Err}^r(t_r)$: error rate among auto-accepted documents.
- Set business costs (from finance/compliance):
  - $C_*\text{human}^r$: cost per human review (high-risk often uses more senior staff).*
  - $C_*\text{err}^r$: expected loss per misclassification at that risk level (regulatory, legal, rework, etc.).[125][126]*

Then for volume V_r docs per period at risk level r:

- Reviews: $V_*\text{rev}^r = V_r(1 - p*_{\text{auto}}{}^r(t_r))$, cost $C_*\text{rev}^r = V*_{\text{rev}}{}^r C_*\text{human}^r$.*
- Errors: $E^r = V_r p_*\text{auto}^r(t_r)\text{Err}^r(t_r)$, cost $C*_{\text{err}}{}^r = E^r C_*\text{err}^r$.[127]*
- Infra/API: $C_*\text{infra}^r = V_r \times$ per-call cost (plus any extra calls for high-risk flows).[128]*

Total periodic cost:

$$C_*\text{total} = \sum_r (C^*_{\text{rev}}{}^r + C_*\text{err}^r + C^*_{\text{infra}}{}^r)$$

****Investment required to "reach" a target risk-aware confidence level**** is then the extra spend to move from your current operating point $\{t_r^{\text{current}}\}$ and accuracy to a tighter policy $\{t_r^{\text{target}}\}$ (e.g., higher empirical accuracy at the same automation rate, or higher automation at the same risk). That investment usually comes from:[129130]

- More labelled data in the combinations where rules say "high-risk" but the model error rate is still too high.
- Model and calibration work (fine-tuning, ensembling, better prompts) targeted to those segments.
- Possibly more infra if you decide high-risk items always get a "second opinion" model.[131132133]

Comparing the **annual reduction** in C_*total against this one-off or ramped investment gives you ROI and payback period.*

Example table (per risk band)

Risk level	Threshold	Auto-rate	Accuracy on auto	Human cost per doc	Error cost per error
Low	0.80	92%	95%	Low	Very low
Medium	0.90	75%	97%	Medium	Medium
High	0.97	40%	99.5%	High	Very high

You can then simulate "what if we shift high-risk threshold from 0.97 to 0.99?" and see extra human-review cost vs reduced error cost.

Would it help if I sketch a concrete numeric example using approximate volumes and costs (e.g., per 100k documents per month) to make this more tangible?

***

Factoring in ModernBERT

Think of this as a ****two-model, one policy**** setup: a ModernBERT-style encoder for cheap, calibrated probabilities, and an LLM "on demand" for the hard or high-risk cases, all still governed by your rule-based risk engine.[139140141142]

1. Where ModernBERT fits vs an LLM

Modern BERT-family encoders (e.g., ModernBERT, RoBERTa, domain BERTs) remain very strong for pure classification and are far cheaper per request than LLMs. In recent benchmarks:[143][144][145]

- Fine-tuned BERT-like models often **match or beat** LLM prompting on text classification F1, especially for pattern-driven tasks.[146][147]
- Their inference cost at 1M requests can be ~100× lower and latency 3–10× better than zero/few-shot LLMs.[148][149][150]

So for your cross-domain document classification:

- Use ModernBERT as the primary classifier with a softmax head, giving well-behaved class probabilities out of the box.[151][152]
- Keep the LLM for: cross-domain generalisation, edge cases, explanation, and as a second opinion on high-risk or low-confidence items.

2. Confidence scoring with ModernBERT + LLM

Because ModernBERT already outputs class probabilities, your confidence layer is more straightforward than with a pure LLM, and you can still combine both. ModernBERT side:[153][154]

- Fine-tune ModernBERT on your labeled data; the classifier head yields logits → softmax probabilities per class.
- Use features like: top probability, margin between top-2 classes, entropy, and maybe embedding distance to class prototypes.
- Train a small calibration/confidence model (e.g., logistic regression, isotonic regression) to map these to calibrated correctness probabilities.[155][156]

LLM side:[157][158]

- Use the LLM only on routed cases (e.g., ModernBERT confidence below a threshold, or rule-engine marks as high-risk).
- For those, either:
  - Use logprobs (if exposed) to derive a probability for the chosen label, or
  - Ask the LLM to choose from a fixed label set and then train a separate calibration model on its outputs.[159]
- Optionally, use LLM self-consistency (multiple samples or re-asks) and agreement rate as another uncertainty signal.[160]

You can then define a **combined** confidence score, for example:

- If only ModernBERT is used: $c = c_*$BERT (its calibrated probability).*

- If LLM is consulted as second opinion: combine c_*BERT and c^*_{LLM} (e.g., average, or use a small meta-model that ingests both plus agreement flag) and re-calibrate that meta-score on held-out data.[161162]

3. Rule-based risk + routing across both models

Your rule engine still defines risk; it just also decides **which model(s)** to invoke:

- Rules produce risk level (low/med/high) from content, entities, metadata.[163164]
- Routing policy might look like:
  - Low risk: ModernBERT only; auto-accept if c_*BERT ≥ 0.8, human or LLM optional otherwise.*
  - Medium risk: ModernBERT first; if c_*BERT ≥ 0.9 auto-accept; else send to LLM as second opinion; if disagreement or low combined confidence, send to human.*
  - High risk: Always ModernBERT + LLM; auto-accept only if both agree **and** combined confidence ≥ 0.97; otherwise human review.[165166167]

This keeps your risk logic deterministic and auditable, and uses the LLM only where it adds incremental safety or coverage.

4. Cost analysis with ModernBERT + LLM

ModernBERT radically changes the cost side, because it's cheap to run at scale compared to LLM APIs.[168169170]

Key costs:

- C_*BERT,inf: cost per ModernBERT inference (often tiny if self-hosted; mostly infra + maintenance).[171]*
- C_*LLM,inf: cost per LLM call (API or self-hosted; orders of magnitude higher).[172173]*
- $C_*\text{human}^r$, $C^*_{\text{err}}{}^r$: as before, risk-dependent human and error costs.[174175]

For each risk level r:

- Let $p_*\text{auto,BERT}^r(t_r)$ be the fraction auto-accepted after ModernBERT only, and $p^*_{\text{LLM}}{}^r$ the fraction escalated to LLM (e.g., low confidence or mandatory for high risk).
- Let $\text{Err}_*\text{BERT}^r$ be the error rate when ModernBERT alone is used above threshold, and $\text{Err}^*_{\text{combo}}{}^r$ when ModernBERT+LLM agree above the stricter high-risk threshold.[176177178]

Costs per period:

- ModernBERT infra: $C_*\text{BERT} = V \times C^*_{\text{BERT,inf}}$ (cheap; essentially all docs).[179][180]
- LLM infra: $C_*\text{LLM} = V \times p^*_{\text{LLM}} \times C_*\text{LLM,inf}$.[181][182]*
- Human + error costs as before, but note that using BERT+LLM on only the risky/uncertain subset usually reduces both error rate and LLM spend vs "LLM for everything."[183][184][185]

When you ask "what investment to reach a target risk-aware confidence?", you now have **two** levers:

- Improve ModernBERT (data, fine-tuning, calibration) so more items can be safely auto-accepted without LLM, lowering LLM and human costs.
- Strengthen the LLM or its verification patterns on the high-risk/edge slice only, which is a much smaller volume.[186][187][188]

That tends to give a very favourable Pareto front: fairly small fine-tuning and infra investment in ModernBERT and a focused LLM budget can reach high risk-adjusted accuracy at much lower total cost than an LLM-only strategy.[189][190][191]

Would it be helpful if I outline a concrete numeric scenario comparing "BERT-only", "LLM-only", and "hybrid BERT+LLM+rules" so you can see the cost and confidence trade-offs side by side?

***

---

1. https://arxiv.org/html/2412.14276v1
2. https://arxiv.org/html/2505.18215v1
3. https://arxiv.org/html/2602.06370v1
4. https://simmering.dev/blog/modernbert-vs-llm/
5. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f
6. https://arxiv.org/html/2401.16458v3
7. https://arxiv.org/html/2412.14276v1
8. https://labelyourdata.com/articles/document-classification
9. https://arxiv.org/html/2401.16458v3
10. https://arxiv.org/html/2412.14276v1
11. https://arxiv.org/html/2412.14276v1
12. https://arxiv.org/html/2602.06370v1

13. https://arxiv.org/html/2412.14276v1
14. https://arxiv.org/html/2602.06370v1
15. https://simmering.dev/blog/modernbert-vs-llm/
16. https://arxiv.org/html/2602.06370v1
17. https://arxiv.org/html/2505.18215v1
18. https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation
19. https://simmering.dev/blog/modernbert-vs-llm/
20. https://arxiv.org/html/2505.18215v1
21. https://simmering.dev/blog/modernbert-vs-llm/
22. https://arxiv.org/html/2602.06370v1
23. https://arxiv.org/html/2505.18215v1
24. https://arxiv.org/html/2506.04290v2
25. https://simmering.dev/blog/modernbert-vs-llm/
26. https://arxiv.org/html/2505.18215v1
27. https://arxiv.org/html/2506.04290v2
28. https://arxiv.org/html/2506.04290v2
29. https://arxiv.org/html/2505.18215v1
30. https://arxiv.org/html/2505.18215v1
31. https://arxiv.org/html/2505.18215v1
32. https://arxiv.org/html/2506.04290v2
33. https://arxiv.org/html/2506.04290v2
34. https://arxiv.org/html/2505.18215v1
35. https://simmering.dev/blog/modernbert-vs-llm/
36. https://arxiv.org/html/2505.18215v1
37. https://arxiv.org/html/2602.06370v1
38. https://arxiv.org/html/2412.14276v1
39. https://simmering.dev/blog/modernbert-vs-llm/
40. https://arxiv.org/html/2505.18215v1

41. https://arxiv.org/html/2506.04290v2
42. https://arxiv.org/html/2505.18215v1
43. https://arxiv.org/html/2602.06370v1
44. https://simmering.dev/blog/modernbert-vs-llm/
45. https://arxiv.org/html/2412.14276v1
46. https://simmering.dev/blog/modernbert-vs-llm/
47. https://docs.nevis.net/nevisdetect/nevisDetect-Reference-Guide/Conceptional-Overview/Normalization/Normalization-models/Rule-based-models
48. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
49. https://www.multimodal.dev/post/using-confidence-scoring-to-reduce-risk-in-ai-driven-decisions
50. https://www.appliedai.de/uploads/files/AI-Act-Risk-Classification-Study-EN.pdf
51. https://pmc.ncbi.nlm.nih.gov/articles/PMC11895222/
52. https://www.reddit.com/r/LanguageTechnology/comments/1r6184q/text_categorization_llm_vs_bert_vs_other_models/
53. https://arxiv.org/html/2412.14276v1
54. https://arxiv.org/html/2505.18215v1
55. https://arxiv.org/html/2602.06370v1
56. https://simmering.dev/blog/modernbert-vs-llm/
57. https://arxiv.org/html/2602.06370v1
58. https://simmering.dev/blog/modernbert-vs-llm/
59. https://arxiv.org/html/2412.14276v1
60. https://arxiv.org/html/2505.18215v1
61. https://arxiv.org/html/2412.14276v1
62. https://simmering.dev/blog/modernbert-vs-llm/
63. https://arxiv.org/html/2412.14276v1
64. https://arxiv.org/html/2505.18215v1
65. https://arxiv.org/html/2602.06370v1
66. https://arxiv.org/html/2602.06370v1

67. https://simmering.dev/blog/modernbert-vs-llm/

68. https://arxiv.org/html/2412.14276v1

69. https://arxiv.org/html/2505.18215v1

70. https://simmering.dev/blog/modernbert-vs-llm/

71. https://arxiv.org/html/2602.06370v1

72. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

73. https://arxiv.org/html/2401.16458v3

74. https://arxiv.org/html/2505.18215v1

75. https://arxiv.org/html/2412.14276v1

76. https://arxiv.org/html/2401.16458v3

77. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

78. https://labelyourdata.com/articles/document-classification

79. https://arxiv.org/html/2401.16458v3

80. https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation

81. https://arxiv.org/html/2506.04290v2

82. https://arxiv.org/html/2401.16458v3

83. https://arxiv.org/html/2506.04290v2

84. https://arxiv.org/html/2401.16458v3

85. https://arxiv.org/html/2505.18215v1

86. https://arxiv.org/html/2412.14276v1

87. https://arxiv.org/html/2412.14276v1

88. https://arxiv.org/html/2505.18215v1

89. https://arxiv.org/html/2602.06370v1

90. https://simmering.dev/blog/modernbert-vs-llm/

91. https://arxiv.org/html/2602.06370v1

92. https://simmering.dev/blog/modernbert-vs-llm/

93. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f
94. https://arxiv.org/html/2401.16458v3
95. https://arxiv.org/html/2505.18215v1
96. https://arxiv.org/html/2412.14276v1
97. https://docs.nevis.net/nevisdetect/nevisDetect-Reference-Guide/Conceptional-Overview/Normalization/Normalization-models/Rule-based-models
98. https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation
99. https://arxiv.org/html/2412.14276v1
100. https://arxiv.org/html/2602.06370v1
101. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
102. https://www.multimodal.dev/post/using-confidence-scoring-to-reduce-risk-in-ai-driven-decisions
103. https://www.appliedai.de/uploads/files/AI-Act-Risk-Classification-Study-EN.pdf
104. https://pmc.ncbi.nlm.nih.gov/articles/PMC11895222/
105. https://www.reddit.com/r/LanguageTechnology/comments/1r6184q/text_categorization_llm_vs_bert_vs_other_models/
106. https://alex-jacobs.com/posts/beatingbert/
107. https://arxiv.org/html/2412.14276v1
108. https://arxiv.org/html/2505.18215v1
109. https://arxiv.org/html/2602.06370v1
110. https://arxiv.org/html/2505.18215v1
111. https://simmering.dev/blog/modernbert-vs-llm/
112. https://simmering.dev/blog/modernbert-vs-llm/
113. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f
114. https://arxiv.org/html/2412.14276v1
115. https://arxiv.org/html/2401.16458v3
116. https://arxiv.org/html/2602.06370v1
117. https://arxiv.org/html/2412.14276v1

118. https://arxiv.org/html/2602.06370v1
119. https://arxiv.org/html/2401.16458v3
120. https://arxiv.org/html/2412.14276v1
121. https://simmering.dev/blog/modernbert-vs-llm/
122. https://labelyourdata.com/articles/document-classification
123. https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation
124. https://arxiv.org/html/2602.06370v1
125. https://labelyourdata.com/articles/document-classification
126. https://arxiv.org/html/2602.06370v1
127. https://arxiv.org/html/2602.06370v1
128. https://arxiv.org/html/2412.14276v1
129. https://arxiv.org/html/2506.04290v2
130. https://arxiv.org/html/2602.06370v1
131. https://docs.nevis.net/nevisdetect/nevisDetect-Reference-Guide/Conceptional-Overview/Normalization/Normalization-models/Rule-based-models
132. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
133. https://arxiv.org/html/2412.14276v1
134. https://www.multimodal.dev/post/using-confidence-scoring-to-reduce-risk-in-ai-driven-decisions
135. https://www.appliedai.de/uploads/files/AI-Act-Risk-Classification-Study-EN.pdf
136. https://pmc.ncbi.nlm.nih.gov/articles/PMC11895222/
137. https://www.reddit.com/r/LanguageTechnology/comments/1r6184q/text_categorization_llm_vs_bert_vs_other_models/
138. https://alex-jacobs.com/posts/beatingbert/
139. https://arxiv.org/html/2412.14276v1
140. https://arxiv.org/html/2505.18215v1
141. https://arxiv.org/html/2602.06370v1
142. https://simmering.dev/blog/modernbert-vs-llm/
143. https://arxiv.org/html/2505.18215v1

144. https://arxiv.org/html/2602.06370v1

145. https://simmering.dev/blog/modernbert-vs-llm/

146. https://arxiv.org/html/2412.14276v1

147. https://arxiv.org/html/2505.18215v1

148. https://arxiv.org/html/2602.06370v1

149. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

150. https://arxiv.org/html/2401.16458v3

151. https://arxiv.org/html/2401.16458v3

152. https://simmering.dev/blog/modernbert-vs-llm/

153. https://labelyourdata.com/articles/document-classification

154. https://simmering.dev/blog/modernbert-vs-llm/

155. https://arxiv.org/html/2602.06370v1

156. https://labelyourdata.com/articles/document-classification

157. https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation

158. https://arxiv.org/html/2412.14276v1

159. https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation

160. https://arxiv.org/html/2412.14276v1

161. https://arxiv.org/html/2506.04290v2

162. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

163. https://docs.nevis.net/nevisdetect/nevisDetect-Reference-Guide/Conceptional-Overview/Normalization/Normalization-models/Rule-based-models

164. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf

165. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf

166. https://www.multimodal.dev/post/using-confidence-scoring-to-reduce-risk-in-ai-driven-decisions

167. https://arxiv.org/html/2506.04290v2

168. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

169. https://arxiv.org/html/2401.16458v3

170. https://arxiv.org/html/2602.06370v1

171. https://arxiv.org/html/2401.16458v3

172. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

173. https://arxiv.org/html/2602.06370v1

174. https://www.appliedai.de/uploads/files/AI-Act-Risk-Classification-Study-EN.pdf

175. https://pmc.ncbi.nlm.nih.gov/articles/PMC11895222/

176. https://simmering.dev/blog/modernbert-vs-llm/

177. https://arxiv.org/html/2505.18215v1

178. https://arxiv.org/html/2506.04290v2

179. https://arxiv.org/html/2602.06370v1

180. https://arxiv.org/html/2401.16458v3

181. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

182. https://arxiv.org/html/2602.06370v1

183. https://arxiv.org/html/2506.04290v2

184. https://arxiv.org/html/2602.06370v1

185. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

186. https://simmering.dev/blog/modernbert-vs-llm/

187. https://arxiv.org/html/2506.04290v2

188. https://arxiv.org/html/2602.06370v1

189. https://arxiv.org/html/2401.16458v3

190. https://arxiv.org/html/2602.06370v1

191. https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f

192. https://www.reddit.com/r/LanguageTechnology/comments/1r6184q/text_categorization_llm_vs_bert_vs_other_models/

193. https://alex-jacobs.com/posts/beatingbert/

194. https://blog.gopenai.com/llms-vs-deterministic-logic-overcoming-rule-based-evaluation-challenges-8c5fb7e8fe46

195. https://www.linkedin.com/posts/vishaljani_aileadership-machinelearning-nlp-activity-7426315146135949312-ae4P

196. https://www.sciencedirect.com/science/article/pii/S0010482525010169

197. https://belitsoft.com/how-much-does-it-cost-to-develop-ai-classification-system